

PROJECT REVIEW – 3

Smart Restaurant Ordering System

Team 20 | Section 10

Course: Database Systems Engineering and Distributed Backend Development (25CS1302E)

Academic Year: 2026–2027

2520090141 – R. Nishanth

2520090050 – T. Shanmukha Sagar

2520080032 – S. Gopal

Project Overview

What it does

A database-driven web application that lets customers view the menu, select food items, choose quantities and place orders. Restaurant staff can manage menu items and update order status.

Why it is needed

Manual ordering can cause incorrect orders, delays, difficulty in tracking orders and extra workload for staff. A centralized system keeps restaurant data organized and consistent.

Main functionality

Menu browsing • Order placement • Customer records • Order tracking • Billing • Menu management • Status updates

Technologies

Frontend: HTML, CSS, JavaScript • Backend: Java • Database: MySQL • Connectivity: JDBC • Tools: VS Code / IntelliJ IDEA, MySQL Workbench

Front-End Development

Designed with HTML, CSS and JavaScript

User Interface

Responsive pages for customers and restaurant staff

Navigation

Move between menu, cart/order, billing and management pages

Forms

Capture customer details, item quantities and order information

Data Display

Show menu items, prices, order status and bill information

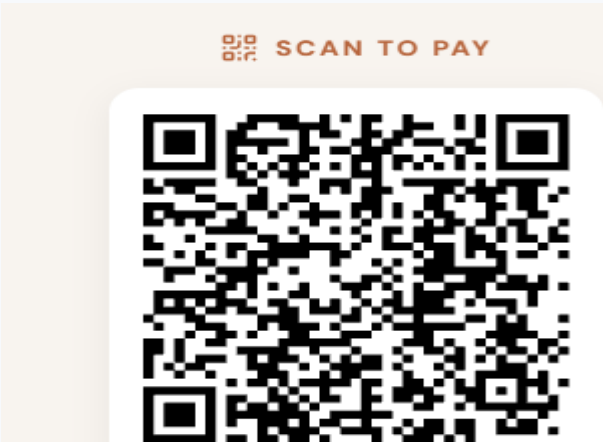
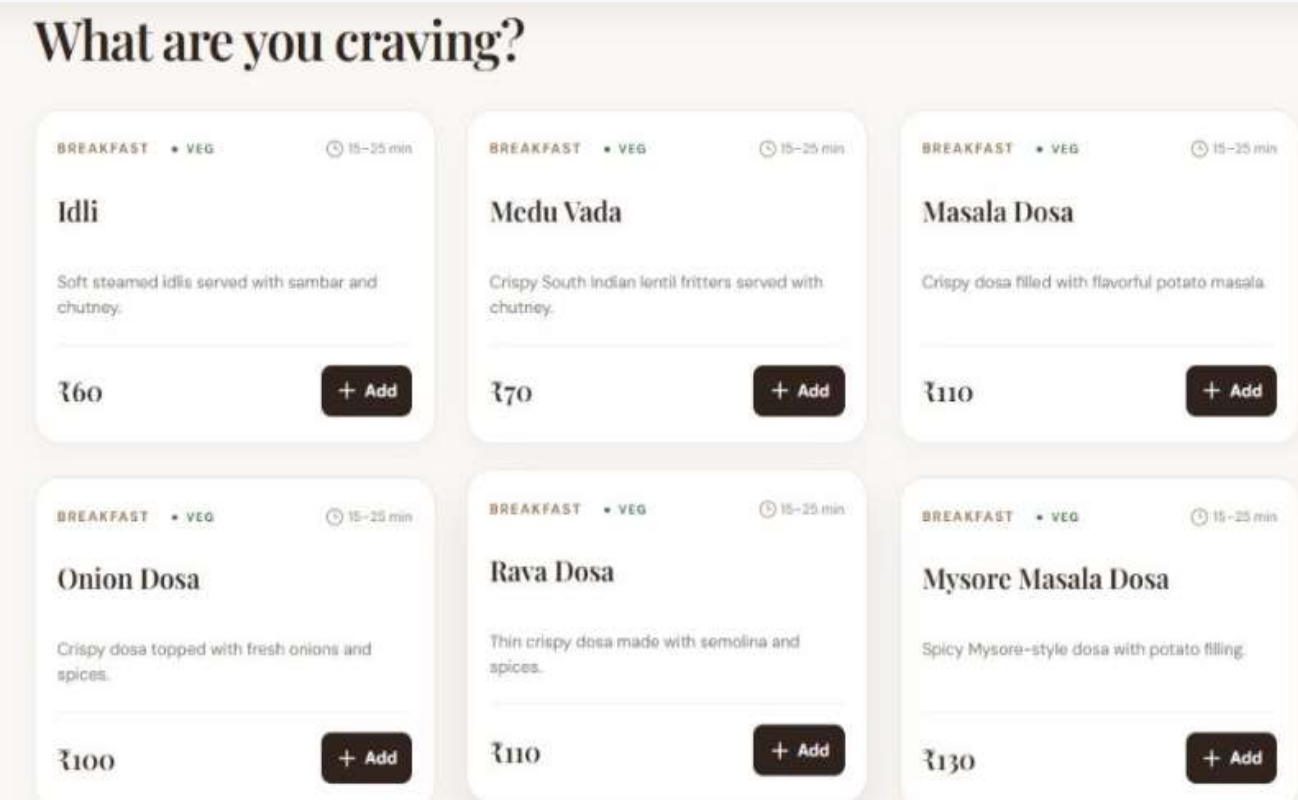
Validation

Check required fields, quantities and basic input correctness

Dynamic Interaction

JavaScript updates totals and page content based on user actions

Frontend Modules / Features

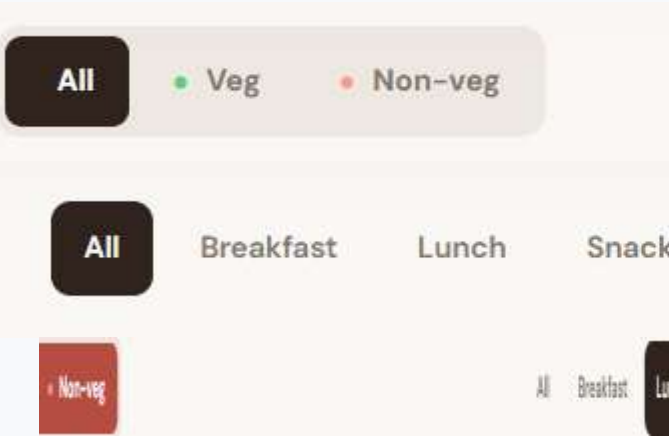
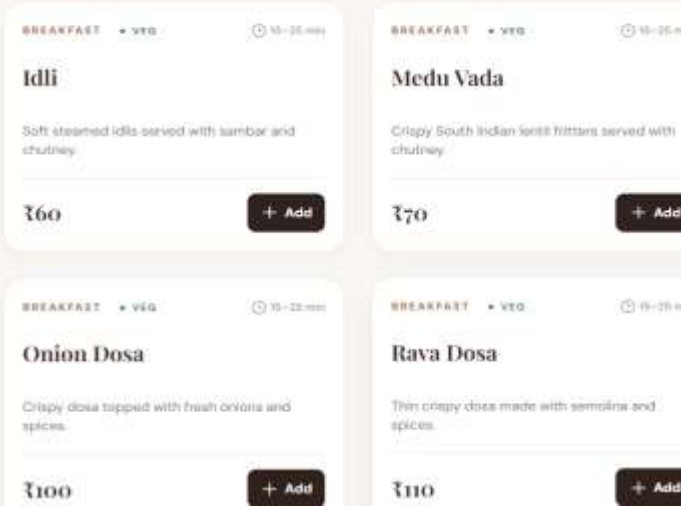


Frontend Screenshots

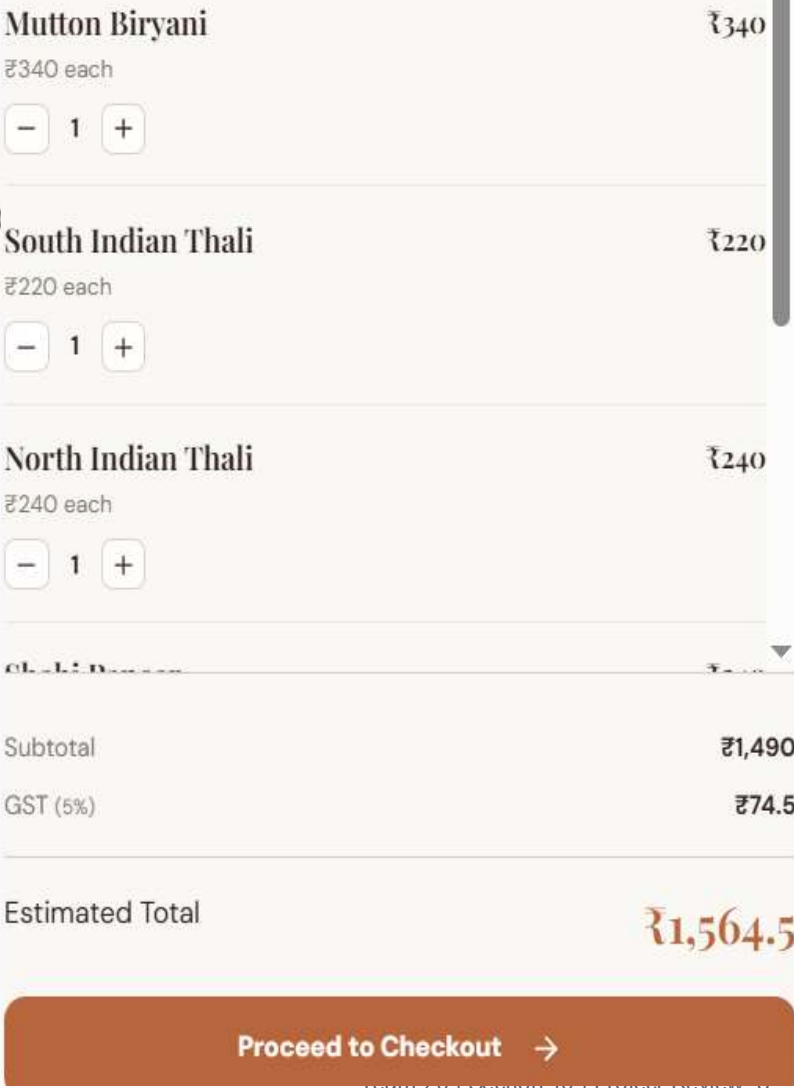
Replace the placeholders below with your project screenshots



What are you craving?



Your Order Basket



Frontend–Backend–Database Flow



- Customer or staff actions in the browser generate requests and form data.
- The Java backend processes the request and uses JDBC to execute SQL queries.
- MySQL stores customer, menu, order and billing data in related tables.
- Results are returned to the application and displayed to the user.

Database Design

Relational Approach

Restaurant information is organized in linked tables instead of repeating the same data in multiple places.

Purpose

Store customers, menu items, orders, order details, billing information and order status.

Data Consistency

Primary keys, foreign keys and constraints help keep records valid.

Efficient Retrieval

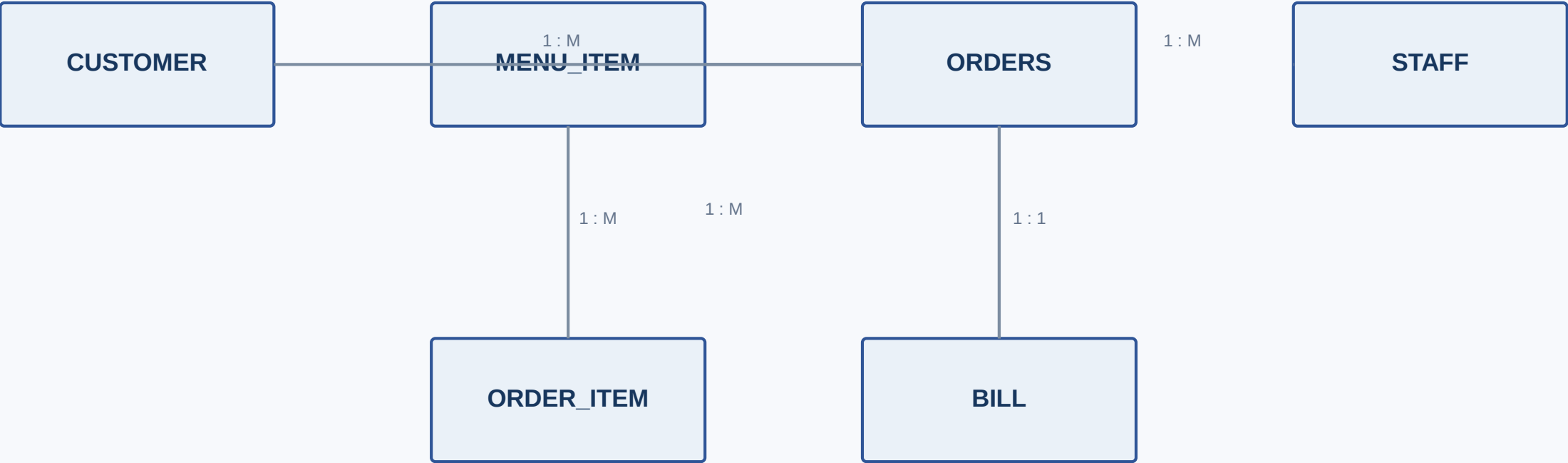
SQL queries and JOINS connect related information for orders and bills.

Suggested database name: smart_restaurant_ordering

Core entities: CUSTOMER • MENU_ITEM • ORDERS • ORDER_ITEM • BILL • STAFF

ER Diagram / Database Schema

Proposed schema based on the project abstract



CUSTOMER places ORDERS • ORDERS contain ORDER_ITEM records • ORDER_ITEM references MENU_ITEM • BILL is generated for an order • STAFF updates menu/order status

Tables and Their Purpose

Table	Purpose
customer	Stores customer details used while placing orders
menu_item	Stores food item name, category, price and availability
orders	Stores each order, customer reference, total and current status
order_item	Stores items and quantities belonging to an order
bill	Stores billing details and total amount for an order
staff	Stores staff/admin account details for restaurant management

Keys and Relationships

Primary Key

Uniquely identifies each record, such as `customer_id`, `item_id` and `order_id`.

Foreign Key

Links related tables, for example `orders.customer_id` referencing `customer.customer_id`.

Referential Integrity

Prevents records from referring to customers, menu items or orders that do not exist.

Important relationships

- CUSTOMER 1 → M ORDERS
- ORDERS 1 → M ORDER_ITEM
- MENU_ITEM 1 → M ORDER_ITEM
- ORDERS 1 → 1 BILL
- STAFF can manage multiple menu items / order updates

Database Operations (CRUD)



CREATE

Add customers, menu items,
new orders and billing
records



READ

View menu, customer details,
orders, order status and bills



UPDATE

Modify menu price/availability
and update order status



DELETE

Remove obsolete menu or
incorrect records when
allowed

Sample SQL Queries

Menu Query

1. View all available menu items

```
SELECT * FROM menu_item WHERE availability = 1;
```

Order Query

2. View one customer's orders

```
SELECT * FROM orders WHERE customer_id = ?;
```

JOIN Query

3. Order details with menu items (JOIN)

```
SELECT o.order_id, m.item_name, oi.quantity, m.price  
FROM orders o  
JOIN order_item oi ON oi.order_id = o.order_id  
JOIN menu_item m ON m.item_id = oi.item_id  
WHERE o.order_id = ?;
```

Current Implementation / Progress

Update the status below before your presentation

Module	Status
Frontend pages	Add your current status
Menu display	Add your current status
Order placement	Add your current status
Java backend	Add your current status
JDBC connectivity	Add your current status
MySQL tables	Add your current status
CRUD operations	Add your current status
Testing	Add your current status

Conclusion and Next Steps

The Smart Restaurant Ordering System centralizes menu, customer, order and billing information in a relational MySQL database. It is designed to reduce manual ordering effort, minimize errors and make order tracking more organized.

Next Steps

- Complete and connect all UI pages with the Java backend
- Validate customer input and order quantities
- Improve order-status handling and error messages
- Test CRUD operations and SQL constraints with sample data
- Add your final project screenshots and ER/database screenshots to this deck

Literature review Hyper Link

[Literature review](#)

THANK YOU

Smart Restaurant Ordering System

R. Nishanth • T. Shanmukha Sagar • S. Gopal